

Reviewer Pack — Minimal Order of p-Adic Units Bounds Determinant Depth of Ci...

Entry a8323471011a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 19:01:45 UTC

Minimal Order of p-Adic Units Bounds Determinant Depth of Circuits

Entry ID: a8323471011a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 19:01:45 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (p-Adic Units) **Field B** (complexity object): Circuit
Complexity: Determinant Depth

Statement:

[‘For all circuits C with determinant depth $d(C) = n$, there exists a set S of p-adic units with minimal order $o(S)$ such that $o(S) \leq \sqrt{n}$.’, ‘For any circuit C with determinant depth $d(C) = n$ and any set S of p-adic units, if $o(S) > \sqrt{n}$, then there exists another set S' with $o(S') \leq o(S)$ and a subcircuit $C' \subseteq C$ such that $\det(C') \neq 0$.’]

Rationale (proposer’s reasoning):

[‘The use of p-adic units in complexity theory has been limited to the study of error correcting codes and number-theoretic algorithms. However, p-adic units are known for their properties regarding divisibility and minimal order, which may offer new insights into circuit depth measures.’, ‘If a connection between the minimal order of p-adic units and determinant depth can be established, it could potentially lead to new algorithms or lower bounds for determinant depth circuits, which is an area with few known results.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 298d14a84c5e4a81

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each circuit C with determinant depth $n \leq 40$, if the minimal order of p-adic units $o(S)$ is less than or equal to $\sqrt{n}$ for all sets S used to represent $\det(C)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic units AND determinant depth - order of p-adic units in circuit complexity AND determinant depth - minimal order bound for p-adic units in circuits with determinant depth

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/2408.00810v3>] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [<http://arxiv.org/abs/hep-th/9410058v3>] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [<http://arxiv.org/abs/2308.04364v3>] Turing patterns in a p-adic FitzHugh-Nagumo system on the unit ball

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError("Inverse doesn't exist")
```

```

    else:
        return x % m

def generate_random_circuit(n):
    circuit = []
    for _ in range(n):
        gate_type = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, 1) for _ in range(random.randint(2, 4))]
        circuit.append((gate_type, inputs))
    return circuit

def evaluate_circuit(circuit):
    stack = []
    for gate_type, inputs in reversed(circuit):
        if gate_type == 'AND':
            result = all(inputs)
        elif gate_type == 'OR':
            result = any(inputs)
        stack.append(result)
    return stack.pop()

def find_minimal_order(p_adic_units):
    p_adic_units = sorted(p_adic_units)
    for i in range(1, len(p_adic_units) + 1):
        if all(p_adic_units[j] % p_adic_units[i] == 0 for j in range(i)):
            return i
    return len(p_adic_units)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    circuit = generate_random_circuit(n)
    p_adic_units = [random.randint(2, 100) for _ in range(10)]

    try:
        result = evaluate_circuit(circuit)
        o_S = find_minimal_order(p_adic_units)
        sqrt_n = math.sqrt(n)

        if o_S <= sqrt_n:
            conjecture_holds = True
            counterexample = ""
        else:
            conjecture_holds = False
            counterexample = "o(S) >  $\sqrt{n}$ "
    except Exception as e:
        return {
            "metric_name": "conjecture_holds",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": str(e)
        }

    return {

```

```

        "metric_name": "o(S)",
        "metric_value": o_S,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"o(S) > \n\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

index out of range'}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'o(S)', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'conjecture_holds', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_893d54ba.py", line 118, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_893d54ba.py", line 118, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data that could confirm or falsify the conjecture.
| next: Review and debug the test code to ensure it can complete without crashing and provide conclusive results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11777
2	preregistration	ollama_remote	glm4:latest	0	0	5981
3	novelty	ollama_remote	glm4:latest	0	0	4519
4	novelty	ollama_remote	glm4:latest	0	0	5991
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23598
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13496
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13031
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10911
9	judge	ollama_remote	glm4:latest	0	0	8593

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97898 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a8323471011a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a8323471011a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a8323471011a.tar.gz` (if generated)